

البنية الداخلية لمحاكي المريض

واجهة خلفية مبنية بالكامل من الصفر بإطار FastAPI لتدريب الأطباء على حالات مرضية محاكاة — مبنية ببنية نظيفة صارمة (Ports & Adapters)، مدعومة بقاعدة بيانات SQLite حقيقية، ومطوّرة ميزة تلو الأخرى: محاكاة مريض تعتمد على المحادثة مدعومة بنموذج لغوي محلي، تقييم بأسلوب OSCE، وخط أنابيب صوتي كامل. يشرح هذا التقرير كيف بُني هذا كله، ولماذا بُني بهذه الطريقة تحديداً، وكيف يتحرك الطلب فعلياً عبره.

faster-whisper

Qwen2.5-0.5B

SQLite • aiosqlite

SQLModel + Alembic

FastAPI

pytest + httpx

gTTS

8

منفذ قابل للتبديل
(وهمي/حقيقي)

52

اختبار ناجح

14

نقطة نهاية (13 REST و1
(WS

4

طبقات معمارية

الاختبار والتحقق

مبادئ التصميم

مرجع ال API

سير عمل الطبيب

الميزات بالترتيب

مسار الطلب

البنية المعمارية

لمحة عامة

الإعداد

المنتج: يفتح الطبيب سيناريو سريريًا، يحاور مريضًا محاكي (نصًا أو صوتًا)، يطلب فحوصات تشخيصية، يجيب عن أسئلة اختبار بأسلوب OSCE، ثم يحصل أخيرًا على تقييم منظم للمواجهة بأكملها. كل ما لا يُفترض أن يعرفه الطبيب مسبقًا — التشخيص الفعلي، مفتاح الإجابات، نتائج الفحوصات — يبقى على الخادم حتى يكشفه الإجراء المقابل ضمن الجلسة.

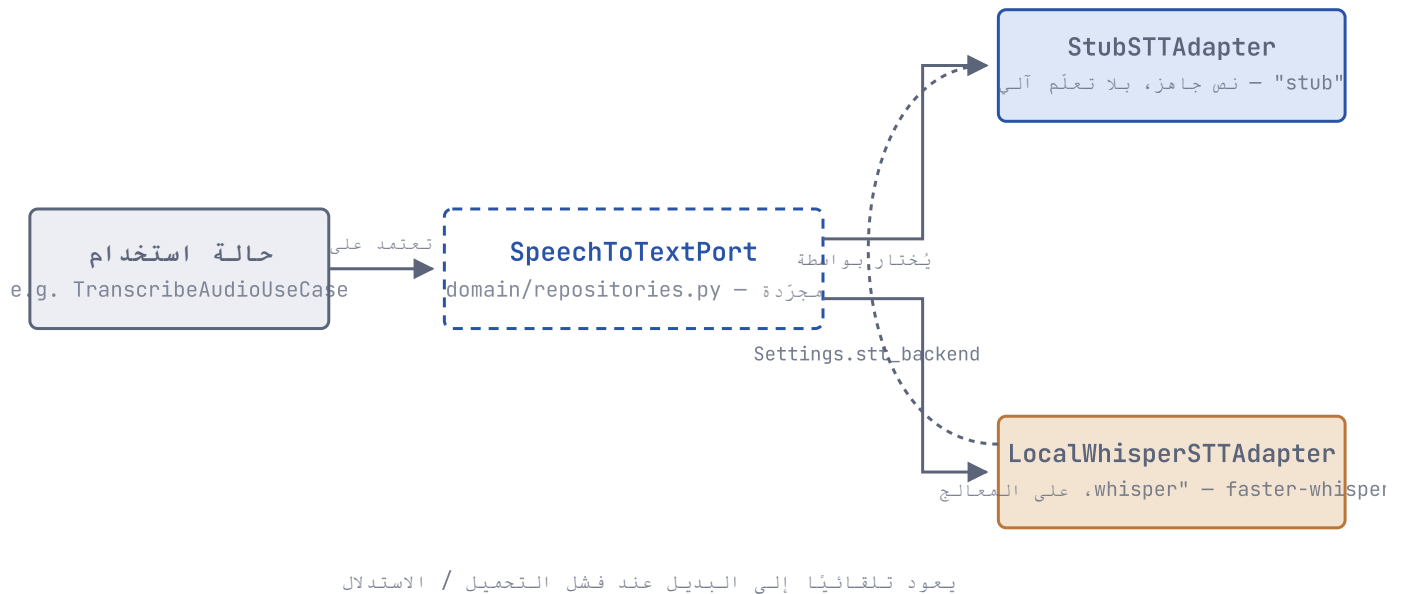
ما كان سيصبح ملفات JSON متفرقة وقواميس في الذاكرة أصبح بدلاً من ذلك قاعدة بيانات SQLite حقيقية، لا يُوصل إليها إلا عبر واجهة خلفية مُقسّمة إلى طبقات وقابلة للاختبار:

- **الحفظ (Persistence)** — نماذج جداول SQLAlchemy، ومحرك غير متزامن، وهجرات Alembic كمصدر وحيد للحقيقة بخصوص المخطط (لا استدعاء لـ `create_all()` في أي مكان).
- **منطق النطاق (Domain)** — كيانات Python صرفة ومنافذ (ports) مجرّدة، بلا أي معرفة بـ HTTP أو SQL أو أي مكتبة تعلّم آلي.
- **ذكاء قابل للتوصيل** — كل قدرة "ذكية" (ردود المريض، التقييم، تحويل الكلام إلى نص، تحويل النص إلى كلام) تقف خلف واجهة تجريدية لها تنفيذ وهمي مجاني وتنفيذ حقيقي بنموذج محلي، قابلان للتبديل بإعداد واحد فقط.
- **الصوت من طرف إلى طرف** — ارفع الصوت أو ابثّه، واحصل على نص مفرّغ، ودورة محادثة محفوظة، وكلامًا مُصنَّعًا في المقابل.

02 البنية المعمارية

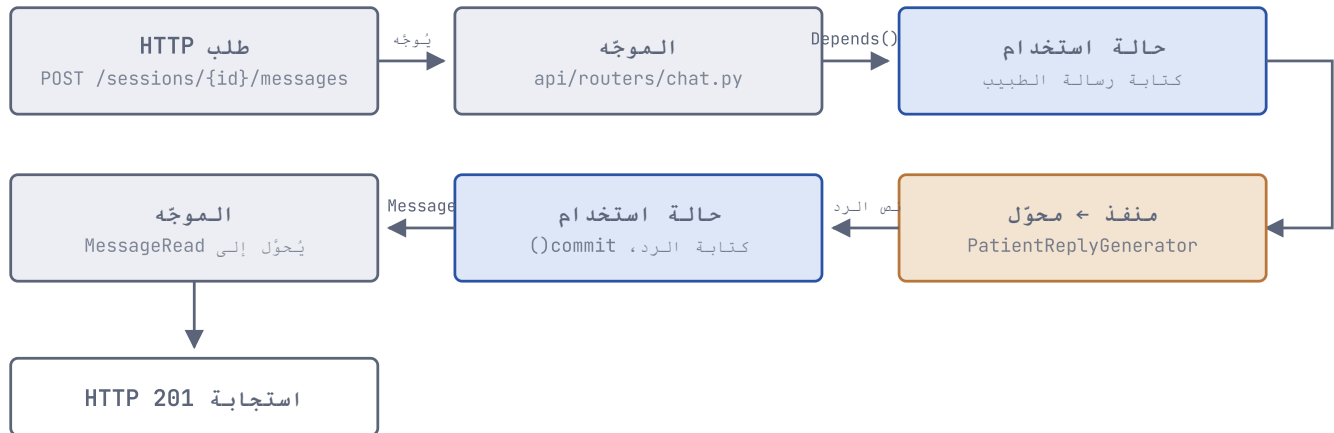
أربع طبقات، باعتماد صارم يتجه إلى الداخل: `api` و `infrastructure` تعتمدان على `application` و `domain` — لا العكس أبدًا. اختبار مخصص (`test_architecture_boundaries.py`) يحلّل كل ملف ضمن `domain/` و `application/` باستخدام وحدة `ast` في بايثون، ويُفشل البناء فور استيراد أحدها لـ `SQLModel` أو `FastAPI` أو `SQLAlchemy` أو أي مكتبة تعلّم آلي/صوت. هذه ليست تعليقًا يعد بالانضباط — بل قاعدة مُنفّذة في كل تشغيل للاختبارات.

- `app/domain/` — `entities.py` (كيانات `dataclasses` صرفة)، `repositories.py` (واجهات مستودعات مجرّدة، وواجهات "المنافذ" الأربعة أدناه أيضًا)، `exceptions.py`.
- `app/application/use_cases/` — صنف واحد لكل إجراء يواجه المستخدم، يُحقّق في مُنشئه بـ `AbstractUnitOfWork` وأي منفذ (منافذ) يحتاجها.
- `app/infrastructure/` — نماذج `SQLModel`، والمحرك غير المتزامن، وتنفيذات المستودعات الفعلية، وكل محوّل منفذ وهمي أو حقيقي.
- `app/api/` — راوترات `FastAPI` رقيقة: تُحلّل الطلب، تستدعي حالة استخدام عبر `Depends()`، وتُحوّل الناتج إلى مخطط `Pydantic`. بلا أي منطق عمل.



الشكل 1. منفذ واحد كمثال تمثيلي (**SpeechToTextPort**) – ونفس الشكل يتكرر لـ **PatientReplyGenerator** و **EvaluationGenerator** و **TextToSpeechPort** : حالة الاستخدام تعتمد فقط على الواجهة المجردة، وقيمة في **Settings** تختار المحوّل الفعلي وقت الطلب، والمحوّل الحقيقي لا يدع فشل التحميل/الاستدلال يصل أبدًا إلى المستدعي – بل يُسلّم الأمر بصمت إلى النسخة الوهمية بدلًا من ذلك.

تتبع طلب حقيقي واحد - `POST /sessions/{id}/messages` - عبر كل طبقة يمر بها، بالترتيب. فحوصات "غير موجود" التي تُجريها كل حالة استخدام أولاً حُذفت هنا للوضوح؛ راجع `post_message.py` للمنطق الكامل.



الشكل 2. طلب واحد، أربع عبورات للطبقات ذهابًا وإيابًا: `api` ← `application` ← `infrastructure` (المنفذ/المحوّل) ← عودة عبر `application` ← `api`. حالة الاستخدام هي المكان الوحيد الذي يتحدث فيه الكود مع قاعدة البيانات (عبر `UnitOfWork` المحقون) ومع المنفذ في الاستدعاء نفسه - الراوترات والمحوّلات لا تتحدث مع بعضها مباشرة أبدًا.

خمس مراحل، كل واحدة أُطلقت باختباراتها الخاصة وتم التحقق منها مقابل خادم فعلي قبل الانتقال للتالية.

01 الأساس: الواجهة الخلفية المُقسّمة نفسها

الهيكل الكامل للبنية النظيفة، من الصف: 8 جداول SQLAlchemyUnitOfWork خلف هجرات Alembic، ومحرك غير متزامن، وتنفيذ مستودع واحد لكل تجميعية ملفوف في SQLAlchemyUnitOfWork، و10 نقاط نهاية REST تغطي السيناريوهات والجلسات والمحادثة وطلب الفحوصات وإجابات الاختبار ومراجعة الجلسة — كل ذلك بقاعدة صارمة لعدم التسريب (نتائج الفحوصات، ومفاتيح الإجابات، وصحة الإجابات لا تُعاد أبدًا قبل الإجراء المفترض أن يكشفها).

`seed.py``UnitOfWork``Alembic``SQLModel`

قرار بدأ رد المحادثة كنموذج بديل ثابت الصدى — كان النموذج الحقيقي دومًا مسألة لاحقة ومنفصلة.

02 مريض محلي حقيقي — Qwen2.5-0.5B-Instruct

استبدال النموذج البديل للمحادثة بنموذج لغوي محلي حقيقي: transformers مع torch على المعالج فقط (CPU)، يُحمّل بتكاسل مرة واحدة لكل عملية ويبقى مخزنًا، ويُبقيه موجّه النظام ضمن الشخصية وباللغة العربية بدقة. كل نمط فشل (لا اتصال إنترنت عند أول تنزيل، نفاد الذاكرة، اعتماديات ناقصة) يعود إلى رد بديل موسوم بوضوح بدلاً من تعطل الطلب.

`PatientReplyGenerator``torch (CPU)``transformers`

ملاحظة صادقة بحجم 0.5 مليار معامل، يخرج النموذج أحيانًا عن الشخصية أو يسترسل بلا ترابط — نموذج حقيقي وعامل وصغير فعليًا، وليس حوارًا بجودة إنتاجية.

منفذ جديد `EvaluationGenerator` ونقطة نهاية `POST /sessions/{id}/evaluate`: يُقيم
المواجهة مقابل `gold_standard` الخاص بالسيناريو، مع تفصيل لكل معيار بنجاح/فشل. يُؤلّد عند الطلب
فقط ولا يُخزّن أبدًا — استدعه مرة أخرى بعد أن يفعل الطبيب المزيد ضمن الجلسة للحصول على قراءة محدّثة.

`stub_evaluator.py`

`EvaluationGenerator`

الاتفاقية أولى من الخطأ طلبت الخطأ رفع `ValueError` مجرّد عند غياب المعيار المرجعي؛ نُقذ بدلاً من ذلك كـ
`MissingGoldStandardError(DomainError)` حقيقي، ليمر عبر نفس تحويل 400 المركزي
الذي تستخدمه كل أخطاء النطاق الأخرى.

بيانات تمهيدية غنية + تغطية كاملة من طرف إلى طرف

مجموعة سيناريوهات عربية واقعية (`seed_data/`) — التهاب الزائدة الدودية بمعيار مرجعي، والتهاب المعدة
بلا معيار، وثلاث تصنيفات فحوصات، وثمانية فحوصات بنتائج معقولة سريريًا، وأسئلة OSCE لكلا السيناريوهين
— بالإضافة إلى حزمة اختبار شاملة (E2E) تمر بكل نقاط REST الإحدى عشرة في تدفق واحد، مزروعة عبر دوال
سكربت التزريع/الفعالية (فتُصبح بذلك اختبار ارتداد لسكربت التزريع نفسه أيضًا). سكربت تشغيل
`scripts/smoke_test.py` ودليل عبر `curl` يغطيان الاختبار اليدوي.

`smoke_test.py`

`test_all_endpoints_e2e.py`

`seed_data/`

خط أنابيب صوتي: تحويل الكلام إلى نص والعكس

منفذان جديدان (`TextToSpeechPort` ، `SpeechToTextPort`) وثلاث نقاط نهاية: `POST /transcribe`، و `POST /sessions/{id}/chat-voice` (صوت يدخل ← نص مفرّغ + دورة محادثة محفوظة + صوت رد مُصنَّع يخرج)، و `WS /ws/voice` لنفس التدفق عبر `WebSocket`، نطق واحد لكل إطار ثنائي. تحويل الكلام إلى نص محلي عبر نموذج `faster-whisper` "tiny" مُجَبَّر على العربية؛ وتحويل النص إلى كلام عبر `gTTS`. `ProcessVoiceChatUseCase` يستدعي `PostMessageUseCase` مباشرة بدلاً من تكرار منطق حفظ المحادثة — المكان الوحيد في هذا المشروع الذي تعتمد فيه حالة استخدام على حالة استخدام أخرى بدلاً من الاعتماد على المنافذ فقط.

WebSocket

gTTS

faster-whisper

ملاحظة صادقة `gTTS` يستدعي نقطة تحويل النص إلى كلام في ترجمة جوجل عبر الشبكة لكل طلب — رغم مطابقته لتسمية "Local*Adapter" في هذا المشروع، فهو ليس محركاً يعمل دون اتصال. تم النظر في `piper-tts` (محرك يعمل فعلياً دون اتصال) وتم تجاوزه: يحتاج `espeak-ng`، وهي اعتمادية نظام لا يمكن تثبيتها عبر `pip` وحدها.

الاتفاقية أولى من الخطأ حُدِّد `/chat-voice` كمسار مسطح في الخطة؛ نُفِّذ بدلاً من ذلك كـ `/sessions/{id}/chat-voice`، مطابقاً لكل تعديل آخر مرتبط بجلسة في الـ `API`.

ما يحدث فعليًا، من البداية للنهاية، عبر كل ما سبق:

1 تصفّح السيناريوهات

GET /scenarios الاسم فقط؛ تبقى الحالة والمعيّار المرجعي مخفيين.

2 ابدأ جلسة

POST /sessions يحصل على معرّف جلسة يُستخدم في كل ما يلي.

3 استجوب المريض

POST /sessions/{id}/messages أو مكافئه الصوتي — دورة محادثة في الحالتين، بنفس السجل المحفوظ.

4 اطلب الفحوصات

POST /sessions/{id}/tests → GET /test-categories/{id}/tests النتائج غير مرئية حتى هذا الاستدعاء تحديداً.

5 أجب عن الاختبار

POST /sessions/{id}/answers → GET /scenarios/{id}/questions لا يتسرّب مفتاح الإجابة، ولا تتسرّب الصحة أيضًا؛ إعادة الإجابة تُحدّث السجل السابق.

6 راجع الجلسة

GET /sessions/{id} المكان الوحيد الذي تظهر فيه صحة الإجابة (is_correct).

7 احصل على تقييم

POST /sessions/{id}/evaluate علامة بأسلوب OSCE وملاحظات لكل معيار، تُؤلّد من جديد في كل استدعاء.

كل نقاط النهاية ال14. أمثلة كاملة للطلب/الاستجابة مع curl جاهزة للنسخ موجودة في `docs/api-` `guide.html`.

الطريقة	المسار	ملاحظات
GET	/health	فحص اتصال فعلي بقاعدة البيانات، لا مجرد بقاء العملية
GET	/scenarios	المعرّف والاسم فقط
GET	/scenarios/{id}/questions	الخيارات فقط، بلا مفتاح إجابة
GET	/test-categories	
GET	/test-categories/{id}/tests	القائمة فقط، بلا نتيجة
POST	/sessions	يُعيد session_id
GET	/sessions/{id}	مراجعة كاملة، تشمل الصحة
POST	/sessions/{id}/messages	دورة محادثة نصية (Qwen أو نسخة وهمية)
POST	/sessions/{id}/tests	يطلب فحصًا ويكشف نتيجته
POST	/sessions/{id}/answers	بلا تسريب للصحة؛ يُحدّث السجل السابق
POST	/sessions/{id}/evaluate	علامة OSCE؛ 400 عند غياب gold_standard
POST	/transcribe	يحوّل الصوت إلى نص
POST	/sessions/{id}/chat-voice	يحوّل الصوت إلى نص مفرّغ + رد + صوت الرد
WS	/ws/voice?session_id=...	نفس خط الأنابيب، نطق واحد لكل إطار ثنائي

القواعد التي تتكرر عبر كل ميزة أعلاه، لا الميزة الأولى فقط:

03 • معاملة واحدة، نتيجة واحدة كل حالة استخدام معدّلة تفتح <code>async</code> مع <code>with uow:</code> مرة واحدة وتنفّذ <code>commit</code> مرة واحدة. أي فشل في المنتصف يُلغي التغييرات — لا كتابة جزئية أبدًا.	02 • تدهور رشيق كل محوّل حقيقي يلتقط أخطاءه الخاصة ويعود إلى نظيره الوهمي. لا شيء في هذه الواجهة الخلفية يُسقط طلبًا بسبب فشل تحميل نموذج.	01 • المنافذ والمحوّلات كل قدرة خارجية — رد النموذج اللغوي، التقييم، تحويل الكلام لنص، تحويل النص لكلام — واجهة مجرّدة في <code>domain/</code> لها نسخة وهمية مجانية وتنفيذ حقيقي في <code>infrastructure/</code>، يُختار عبر حقل واحد في <code>Settings</code>.
06 • الاتفاقية أولى من نص الطلب حين تعارضت الصياغة الحرفية لطلب ميزة ما مع نمط قائم فعلاً (مسار مسطّح مقابل تعشيش داخل الجلسة، استثناء مجرّد مقابل هرمية أخطاء النطاق)، انتصرت الاتفاقية القائمة — في كل مرة، وموثّقة في مكانها.	05 • مُنفّذ لا مجرد وعد <code>st_architecture_boundaries.py</code> يفحص إحصائيًا كل ملف ضمن <code>domain/</code> و <code>application/</code> بحثًا عن استيرادات محظورة — أي خرق لحدود الطبقات يُفشل الاختبار الآلي، لا مراجعة الكود فقط.	04 • لا تسريب مبكر نص الحالة، والمعايير المرجعية، ونتائج الفحوصات، ومفاتيح الإجابات، وصحة الإجابات — كل منها يُعاد عبر إجراء واحد فقط، لا قبله أبدًا.

08 الاختبار والتحقق

52 اختبارًا عبر 8 ملفات، جميعها سريعة (~20-30 ثانية) لأن التركيبات المشتركة تجعل كل منفذ تعلّم آلي/صوت يستخدم نسخته الوهمية افتراضيًا:

PYTEST -Q

```
tests/test_all_endpoints_e2e.py      # 2  -- full 11-endpoint flow, real seed_data
tests/test_architecture_boundaries.py # 21 -- layer-boundary enforcement
tests/test_evaluation.py             # 5  -- OSCE evaluation
tests/test_qwen_patient_generator.py # 4  -- real Qwen model, self-skips if unavailable
tests/test_scenarios.py              # 7  -- no-leak rules
tests/test_sessions_flow.py          # 5  -- core session/DB flow
tests/test_voice.py                  # 8  -- STT/TTS ports, REST + both WS scenarios
```

أمران يستحقان الذكر. أولاً، كل اختبار مدعوم بتعلّم آلي (Qwen، Whisper) يُشغّل النموذج/الحقيقي فعليًا ويتخطّى نفسه تلقائيًا — لا يفشل أبدًا — إن لم يتوفر النموذج أو اعتمادياته، فتبقى الحزمة خضراء في البيئات المقيّدة دون خسارة تغطية حقيقية حيث تتوفر. ثانيًا، كل ميزة في هذا التقرير جرى التحقق منها أيضًا مقابل خادم `uvicorn` فعلي يعمل، باستخدام `curl`، ولمسار WebSocket سكربت عميل WS حقيقي — لا الاعتماد على حزمة الاختبار وحدها — قبل اعتبارها منجزة.

09 الإعداد

```
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env

alembic upgrade head
python -m app.infrastructure.seed seed_data

uvicorn app.main:app --reload    # http://127.0.0.1:8000/docs
pytest
```

الإعداد الكامل، والتهيئة، وتفصيل حدود الطبقات موجودة في `README.md`. دليل خطوة بخطوة باللغة العربية
لاختبار كل نقطة نهاية بأمثلة `curl` جاهزة للنسخ موجود في `docs/api-guide.html`.